

CSE 4403: Algorithms

Lecture19+20

Greedy Algorithms

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

What is the Greedy Method?

At each step, make the choice that looks best right now, the locally optimal choice, without reconsidering earlier decisions hoping it leads to a globally optimal solution.

Greedy Choice Property

A globally optimal solution can be reached by choosing the locally optimal option at each step, without revisiting past choices.

Optimal Substructure

An optimal solution to the problem contains within it optimal solutions to its subproblems.

Greedy is fast and simple but it does not work for every optimization problem.

The General Greedy Strategy

1

Define the greedy criterion

Pick a measure that ranks choices : a ratio, a finish time, a frequency.

2

Sort or order the input

Arrange candidates by that criterion, usually in $O(n \log n)$.

3

Iterate and choose

Walk through candidates in order; take each one if it keeps the solution feasible.

4

Never reconsider

Once a choice is made (or rejected), it is final and no backtracking is needed.

Fractional Knapsack

Given n items, each with a weight and a value, and a knapsack of capacity W , choose how much of each item to carry. The items may be broken into fractions to maximize total value without exceeding capacity.

□ Given

Items with (weight, value) pairs; capacity W

□ Goal

Maximize total value carried

□ Constraints

Total weight $\leq W$;
fractions of items allowed

Contrast with 0/1 Knapsack: there, each item must be taken whole or left behind greedy fails and DP is required.

Greedy Strategy & Algorithm

1

Compute ratios

For each item i , compute $\text{value/weight} = v_i / w_i$ “value per unit weight.”

2

Sort descending

Order all items by ratio, highest first, in $O(n \log n)$.

3

Fill greedily

Take whole items in order while they fit in the remaining capacity.

4

Take a fraction

When the next item doesn't fully fit, take just enough of it to fill the knapsack, then stop.

Worked Example

Capacity $W = 50$

Item	Weight	Value	Ratio (v/w)
A	10	60	6.0
B	20	100	5.0
C	30	120	4.0

Maximum Total Value = ?

Worked Example

Capacity $W = 50$

Item	Weight	Value	Ratio (v/w)	Taken
A	10	60	6.0	100% → 60
B	20	100	5.0	100% → 100
C	30	120	4.0	66.7% → 80

Maximum Total Value = ?

Interval Scheduling (Activity Selection)

Given single resource used by n activities, each with a start time and a finish time, select the largest possible set of activities such that no two selected activities overlap.

□ Given

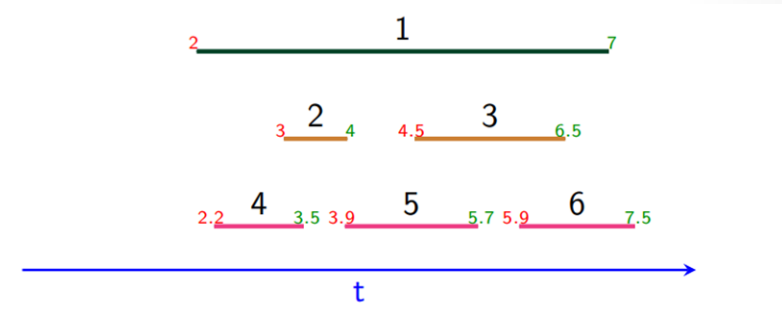
Activities with (start, finish) times

□ Goal

Maximize the number of activities selected

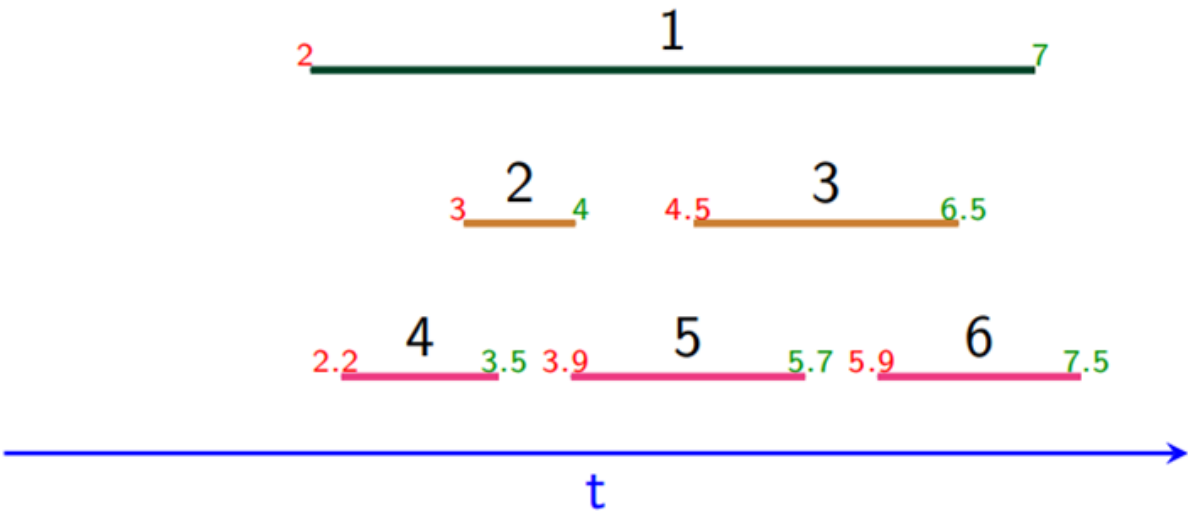
□ Constraint

Two activities are compatible if they don't overlap



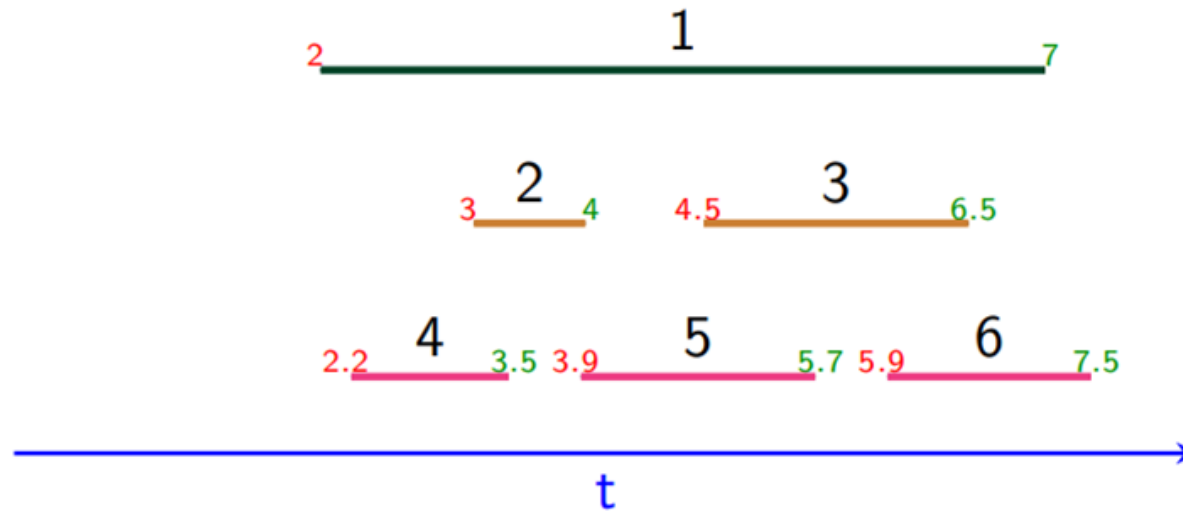
Real-world framing: booking a single lecture hall for the most classes possible, or scheduling one machine to run the most jobs.

Interval Scheduling (Activity Selection)



Greedy Solution Idea

- ❑ Sort the requests
- ❑ Select the requests from the sorted list, that does not overlap with the already selected requests

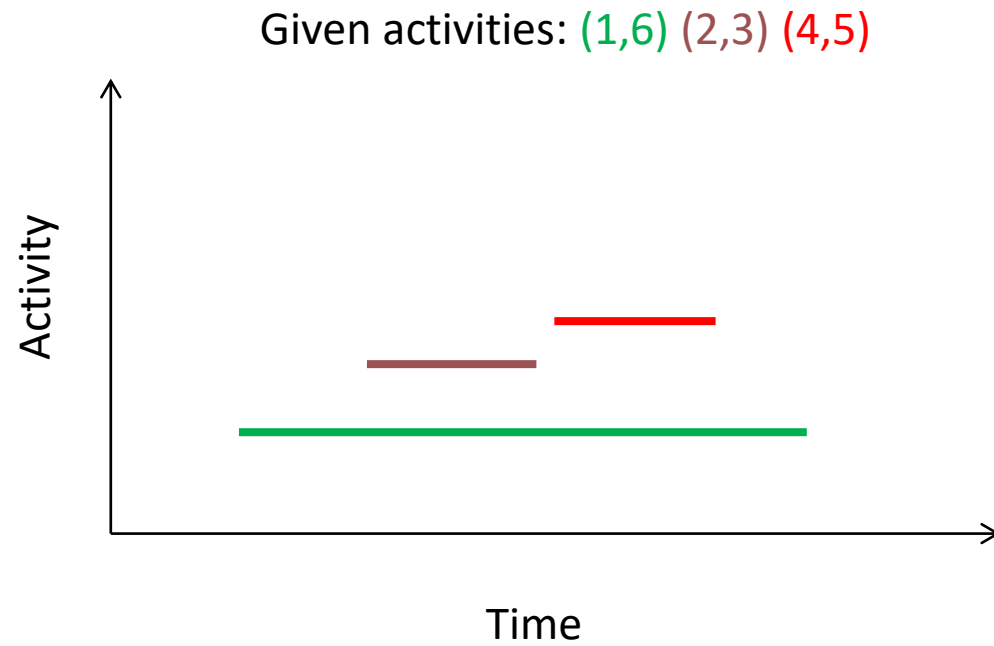


Greedy Solution (Sorting Options)

- ☐ Earlier start time
- ☐ Smallest request
- ☐ Minimum overlap (fewest conflicts)
- ☐ Earlier finish time

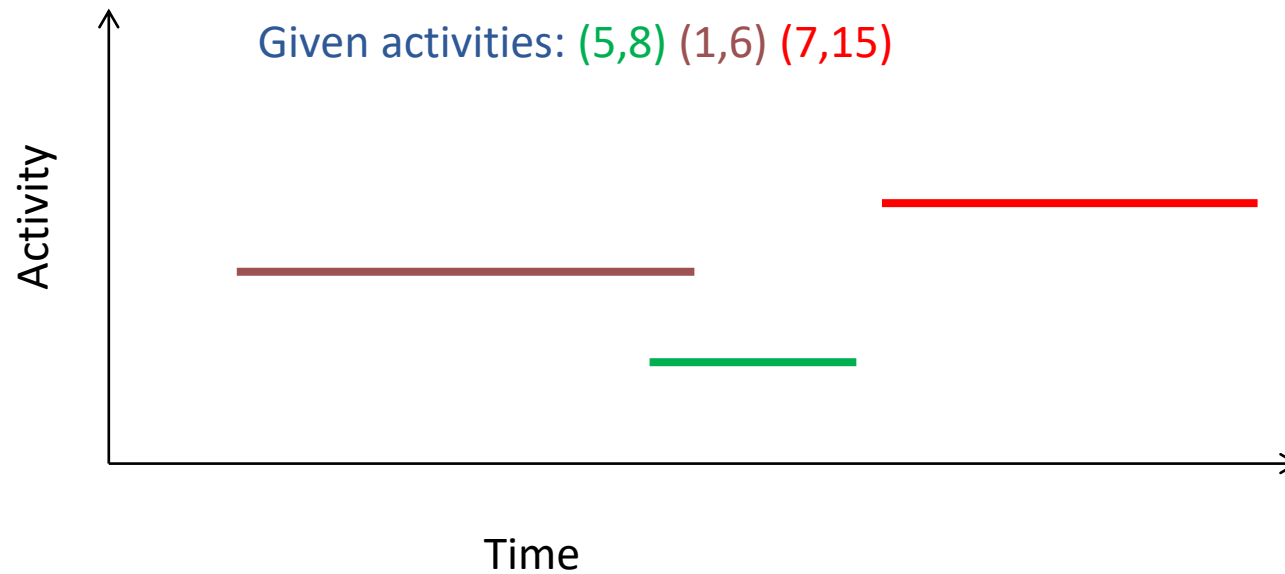
Greedy Solution (Sorting Options)

❑ Earlier start time. Will it work?



Greedy Solution (Sorting Options)

❑ What about smallest activity first?



Greedy Solution (Sorting Options)

☐ Minimum overlaps/conflicts?

Greedy Solution (Sorting Options)

❑ Minimum overlaps/conflicts?



counterexample for earliest start time



counterexample for shortest interval



counterexample for fewest conflicts

Greedy Strategy & Algorithm

1

Sort by finish time

Order all activities by finish time, ascending in $O(n \log n)$.

2

Select the first

Always take the activity that finishes earliest overall.

3

Scan and compare

For each next activity, compare its start time to the finish time of the last selected one.

4

Select if compatible

If $\text{start} \geq \text{last finish}$, select it and update “last finish”; otherwise skip and move on.

Proof of Correctness(1/5) : Setup

Let,

G = the set of intervals selected by the **greedy algorithm**

O = the set of intervals in some **optimal solution**

If $G = O$, then G is already optimal.

If $G \neq O$, there is nothing to prove, so assume that $G \neq O$

Goal: Prove by Contradiction that G and O are actually the same

Proof of Correctness(2/5): Assume G and O differ

Write both solutions as sequences, ordered by finish time:

$$\mathbf{G} = \{ g_1, g_2, \dots, g_m \}$$

$$\mathbf{O} = \{ r_1, r_2, \dots, r_n \}$$

Here, $n \geq m$

Since O is an optimal solution and G is just some valid greedy solution, O can be no smaller than G, hence $n \geq m$.

Proof of Correctness(3/5) : The First Mismatch

Let k be the index of the first interval where the two sequences differ:

$$\mathbf{G} = \{ g_1, g_2, \dots, g_{k-1}, g_k, \dots, g_m \}$$

$$\mathbf{O} = \{ g_1, g_2, \dots, g_{k-1}, r_k, \dots, r_n \}$$

(the first $k - 1$ intervals already agree)

Can we replace r_k with g_k in $\mathbf{O}$ without creating a conflict, and without $\mathbf{O}$ losing its optimality?

Proof of Correctness(4/5) : The Exchange

As both of g_k and r_k is compatible with g_{k-1} , so different start time does not create any conflict.

As, $f_{g_k} \leq f_{r_k}$

So, we can replace r_k with g_k without creating conflict with r_{k+1}

After replacement:

$$G = \{g_1, g_2, g_{k-1}, g_k, \dots, g_m\}$$

$$O = \{g_1, g_2, g_{k-1}, g_k, \dots, r_n\}$$

Proof of Correctness(5/5)

By repeating this process, we can replace all g_i to the set O

$$G = \{g_1, g_2, g_{k-1}, g_k, \dots, g_m\}$$

$$O = \{g_1, g_2, g_{k-1}, g_k, \dots, g_m, \dots, r_n\}$$

Suppose, for contradiction, that **O contains an interval after g_m that doesn't conflict with it.** By definition, **greedy only terminates when no remaining interval is compatible with its last choice.** So such an interval **would have been selected by greedy and would already belong to G ,** contradicting the assumption that it's exclusive to O . Hence no such interval exists, and O cannot extend past G .

So $|G|=|O|$, and we can convert G to O , thus proving the optimality of G

Complexity & Correctness

Greedy choice property

There always exists an optimal solution that includes the activity with the earliest finish time. If some optimal solution excludes it, swapping it in for that solution's first activity can only free up more time, never less.

Optimal substructure

After selecting the earliest-finishing activity, the remaining problem choosing among activities that start after it finishes is the same problem on a smaller input.

$O(n \log n)$

Dominated by the initial sort

$O(n)$

Single linear scan to select

1 pass

No backtracking needed

Job Sequencing

Given n jobs, each with a deadline and a profit, and each taking **exactly one unit of time**, schedule jobs, at most one at a time, to maximize total profit. A job only earns its profit if completed by its deadline.

□ Given

Jobs with (deadline, profit) pairs

□ Goal

Maximize total profit from completed jobs

□ Constraints

One job per time slot; a job must finish by its deadline to count

Real-world framing: a single CPU or worker choosing which one-unit tasks to run before their deadlines expire.

Input: Four Jobs with following deadlines and profits

JobID	Deadline	Profit
-------	----------	--------

a	4	20
---	---	----

b	1	10
---	---	----

c	1	40
---	---	----

d	1	30
---	---	----

Output: Following is maximum profit sequence of jobs

c, a

Input: Five Jobs with following deadlines and profits

JobID	Deadline	Profit
-------	----------	--------

a	2	100
---	---	-----

b	1	19
---	---	----

c	2	27
---	---	----

d	1	25
---	---	----

e	3	15
---	---	----

Output: Following is maximum profit sequence of jobs

c, a, e

Greedy Strategy & Algorithm

1

Sort by profit

Order all jobs by profit, descending, in $O(n \log n)$.

2

Scan in profit order

Take each job in turn, most profitable first.

3

Find the latest free slot

Search for the latest available time slot i with $i < \text{deadline}$.

4

Place or skip

If such a slot exists, schedule the job there; otherwise it earns no profit, so skip it.

Greedy Strategy & Algorithm

```
// Step 2: scan jobs in profit order
for (auto& j : jobs) {
    // Step 3: find the latest free slot i with i <= deadline (and i >= 1)
    for (int i = min(maxDeadline, j.deadline); i >= 1; i--) {
        if (slot[i] == -1) {
            // Step 4: place it
            slot[i] = j.id;
            totalProfit += j.profit;
            jobsScheduled++;
            break;
        }
        // else: slot taken, try the next earlier slot
    }
    // if no free slot was found, the job is simply skipped
}
```

Worked Example

Four Jobs

Job	Deadline	Profit	Rank	Slot Assigned
c	1	40	1	Slot 1
d	1	30	2	— (skip)
a	4	20	3	Slot 4
b	1	10	4	— (skip)

Total Profit = ?